

작은 조각부터 쌓아올리는 전체 시스템 — 개발 첫걸음

함수 한 개 → 서비스 1개 → 서비스들 연결 → 전체 시스템. 아래로 갈수록 커집니다. **파란 글씨 = 쉬운 비유**

1

가장 작은 조각 — 프로그램은 이 3가지로 시작해요

함수 일 하나 하는 코드 (**≈ 레시피 한 단계 "계란 깨기"**) · 예: 비밀번호 맞는지 확인 **데이터 한 줄** 기록 한 건 (**≈ 엑셀 한 행**) · 예: 로그인 시도 1건 login_attempt
요청 한 개 서버에 보내는 부탁 (**≈ 창구에 낸 신청서**) · 예: '로그인' 버튼 클릭

▼ 여러 개를 모으면

2

조각을 모으면 — 뭉치가 됩니다

함수 여러 개 → 한 가지 기능(로직) LoginService **데이터 줄 여러 개** → 표 하나(테이블) (**≈ 엑셀 시트**) login_attempt **테이블 여러 개** → 데이터베이스(DB) (**≈ 서류 캐비닛**) PostgreSQL

▼ 기능 + DB를 한 덩어리로 묶으면

3

하나의 서비스(앱 1개) — ≈ 회사의 '부서' 하나. 자기 일과 자기 서류함을 가짐

요청 받기 Controller → **일 처리 Service** → **DB 저장 Repository** → **전용 DB**
(**≈ 부서의 접수창구 → 실무자 → 서류담당 → 서류함**) 예: customer-service(인증 담당) + customer-db

▼ 이런 부서(서비스)가 여러 개 → 서로 협력해야 함

4

서비스들이 대화하는 법 — 부서(서비스) 4개가 3가지 방법으로 소통

정문 경비 · API Gateway 요청이 오면 **신분증(JWT) 확인** 후 알맞은 부서로 배달 (**≈ 건물 정문 경비**)
전화 · REST 호출 지금 바로 다른 부서에 물어봄 (**≈ 옆 부서에 전화**) **우편함 · Kafka** 사건을 적어두면 다른 부서가 나중에 읽어감 (**≈ 사내 우편함·게시판**)
부서(서비스) 4개 : **인증** customer · **수신** deposit · **여신** loan · **결제** payment (각자 자기 DB)

▼ 사용자 화면부터 감시·배포까지 다 합치면

5

전체 시스템 — 위 조각들이 다 연결된 모습

사용자 → **브라우저·앱 화면 Next.js** → **정문 Gateway** → **부서 4개 + 각 DB** ↔ **우편함 Kafka** → **외부 은행망 KFTC-BOK**
감시 · 모니터링 잘 돌아가나 CCTV·경보 (**≈ 관제실**) Prometheus · Grafana **배포 · CI/CD** 새 버전을 자동으로 설치 (**≈ 건물에 새 설비 교체**) GitHub Actions → Docker

한 줄 정리 — **함수**를 모아 **기능·DB**를 만들고, 그걸 묶어 **서비스(부서)**가 되고, 서비스들을 **Gateway·Kafka**로 연결하면 **전체 시스템**이 됩니다.

작은 것 → 큰 것 : 함수 → 테이블/DB → 서비스 1개 → 서비스 여러 개 연결 → 화면·감시·배포까지 = 전체